

Lecture 11. Laws and induction

Functional Programming

Utrecht University

What does it mean for programs to be equal/equivalent?

Goals

- Use definitions and laws to prove program equalities
- Choose the right reasoning principle:
  - extensional equality for functions;
  - case distinction for non-recursive data;
  - structural induction for recursive data.
- Recognize when a proof needs an auxiliary lemma

Reading: Hutton, Chapter 16 (up to 16.6); lecture notes, Chapter 13

Laws

Mathematical laws

- Pure mathematical functions do not inspect hidden, changing state
  - $2 + 3 = 5$, both in $4 \times (2 + 3)$ and in $(2 + 3)^2$
- A **law** is an equation intended to hold for all values of the stated types
  - Laws let us replace equals by equals inside larger expressions

Examples of laws for integers

$+$ commutes	$x + y = y + x$
$\times$ commutes	$x \times y = y \times x$
$+$ is associative	$x + (y + z) = (x + y) + z$
$\times$ distributes over $+$	$x \times (y + z) = x \times y + x \times z$
0 is the unit of $+$	$x + 0 = x = 0 + x$
1 is the unit of $\times$	$x \times 1 = x = 1 \times x$

Putting laws to good use

Why care about program equivalences?

Putting laws to good use

Why care about program equivalences?

- Laws justify semantics-preserving improvements to **performance**
  - Equal expressions can still take different amounts of time or memory
  - Replace an expensive expression by a cheaper equal one

Putting laws to good use

Why care about program equivalences?

- Laws justify semantics-preserving improvements to **performance**
  - Equal expressions can still take different amounts of time or memory
  - Replace an expensive expression by a cheaper equal one

- Laws also express **correctness**: an implementation has the property we intended

Putting laws to good use

Why care about program equivalences?

- Laws justify semantics-preserving improvements to **performance**
  - Equal expressions can still take different amounts of time or memory
  - Replace an expensive expression by a cheaper equal one

- Laws also express **correctness**: an implementation has the property we intended

In short: optimize without changing meaning, and prove what code does

Equational reasoning by example

```
(a + b)²                          = -- definition of square
(a + b) × (a + b)                 = -- distributivity
((a + b) × a) + ((a + b) × b)     = -- commutativity of ×
(a × (a + b)) + (b × (a + b))     = -- distributivity, twice
(a × a + a × b) + (b × a + b × b) = -- associativity of +
a × a + (a × b + b × a) + b × b   = -- commutativity of ×
a × a + (a × b + a × b) + b × b   = -- definition of square and (2 ×)
a² + 2 × a × b + b²
```

Each theory has its laws

- We have seen laws that deal with arithmetic operators
- During courses in logic you have seen similar laws for logic operators

commutativity of $\wedge$	$x \wedge y = y \wedge x$
associativity of $\wedge$	$x \wedge (y \wedge z) = (x \wedge y) \wedge z$
distributivity of $\wedge$ over $\vee$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$
De Morgan's law	$\neg(x \wedge y) = \neg x \vee \neg y$
Howard's law	$(x \wedge y) \to z = x \to (y \to z)$

A small proof in logic

```
¬((a \/ b) \/ c) -> ¬d
= -- De Morgan's law
(¬(a \/ b) /\ ¬c) -> ¬d
= -- De Morgan's law
((¬a /\ ¬b) /\ ¬c) -> ¬d
= -- Howard's law
(¬a /\ ¬b) -> (¬c -> ¬d)
= -- Howard's law
¬a -> (¬b -> (¬c -> ¬d))
```

- Proofs feel mechanical
  - You apply the “rules” implicit in the laws
  - Possibly even without understanding what $\wedge$ and $\vee$ do
- Always provide a hint why each equivalence holds!

Back to Haskell

- Haskell is referentially transparent
  - Equal expressions can be replaced without changing program behaviour

Back to Haskell

- Haskell is referentially transparent
  - Equal expressions can be replaced without changing program behaviour
- Today we focus on pure, terminating expressions
  - In do, `<-` sequences `IO` actions; it is not assignment

Back to Haskell

- Haskell is referentially transparent
  - Equal expressions can be replaced without changing program behaviour
- Today we focus on pure, terminating expressions
  - In do, `<-` sequences `IO` actions; it is not assignment
- A defining equation is also a rewrite rule

```
double x = x + x
```

  1. The *definition* of a function
  2. A *property* for reasoning
     - Rewrite in either direction whenever the pattern matches

Back to Haskell

- Haskell is referentially transparent
  - Equal expressions can be replaced without changing program behaviour
- Today we focus on pure, terminating expressions
  - In do, `<-` sequences `IO` actions; it is not assignment
- A defining equation is also a rewrite rule

```
double x = x + x
```

  1. The *definition* of a function
  2. A *property* for reasoning
     - Rewrite in either direction whenever the pattern matches
- Definitions and previously proved laws are our rewrite rules

A first example

For `f :: b -> c`, `g :: a -> b`, and `xs :: [a]`,

```
(map f . map g) xs = map (f . g) xs
```

This is not a definition, but a property/law

- The law can be shown to hold for the usual definitions of `map` and `(.)`

A first example

For `f :: b -> c`, `g :: a -> b`, and `xs :: [a]`,

```
(map f . map g) xs = map (f . g) xs
```

This is not a definition, but a property/law

- The law can be shown to hold for the usual definitions of `map` and `(.)`

Why care about this law?

A first example

For `f :: b -> c`, `g :: a -> b`, and `xs :: [a]`,

```
(map f . map g) xs = map (f . g) xs
```

This is not a definition, but a property/law

- The law can be shown to hold for the usual definitions of `map` and `(.)`

Why care about this law?

The right-hand side expresses a single traversal

- Without fusion, the left-hand side constructs an intermediate list
- GHC may sometimes perform this optimization automatically

Relation to imperative languages

The law `map (f . g) = map f . map g` resembles loop fusion

```
var ys = new List<B>();
foreach (var x in xs) { ys.Add(g(x)); }
var zs = new List<C>();
foreach (var y in ys) { zs.Add(f(y)); }
=
var zs = new List<C>();
foreach (var x in xs) { zs.Add(f(g(x))); }
```

Relation to imperative languages

The law `map (f . g) = map f . map g` resembles loop fusion

```
var ys = new List<B>();
foreach (var x in xs) { ys.Add(g(x)); }
var zs = new List<C>();
foreach (var y in ys) { zs.Add(f(y)); }
=
var zs = new List<C>();
foreach (var x in xs) { zs.Add(f(g(x))); }
```

Challenge: which effects could make the two programs observably different?

A few important laws

1. Function composition is associative

```
f . (g . h) = (f . g) . h
```

A few important laws

1. Function composition is associative

```
f . (g . h) = (f . g) . h
```

2. `map f` distributes over `(++)`

```
map f (xs ++ ys) = map f xs ++ map f ys
```

A few important laws

1. Function composition is associative

```
f . (g . h) = (f . g) . h
```

2. `map f` distributes over `(++)`

```
map f (xs ++ ys) = map f xs ++ map f ys
```

   - Allows a large `map` to be split into independent chunks
   - There is a generalization to lists of lists

```
map f . concat = concat . map (map f)
```

A few important laws

1. Function composition is associative

```
f . (g . h) = (f . g) . h
```

2. `map f` distributes over `(++)`

```
map f (xs ++ ys) = map f xs ++ map f ys
```

   - Allows a large `map` to be split into independent chunks
   - There is a generalization to lists of lists

```
map f . concat = concat . map (map f)
```

3. `map` preserves composition (the functor composition law)

```
map (f . g) = map f . map g
```

A few (more) important laws

4. If op is associative and e is its two-sided unit, then for finite lists xs

```
foldr op e xs = foldl op e xs
```

A few (more) important laws

4. If op is associative and `e` is its two-sided unit, then for finite lists `xs`

   ```
   foldr op e xs = foldl op e xs
   ```

5. Under the same conditions, `foldr` on a singleton list is the identity

   ```
   foldr op e [x] = x
   ```

These rules apply to very general functions

- GHC can encode equalities like these as rewrite and fusion rules

Why prove the laws?

- A proof justifies a transformation for every input covered by its assumptions
  - Otherwise an optimization may silently change behaviour
- Testing can find counterexamples; a proof explains why none exist
- Proofs can be wrong too
  - Mechanically checked proofs can provide additional assurance

Proving is like programming

1. Proposition = specification
2. Proof = implementation
3. Lemma = helper function

Proving is like programming

1. Proposition = specification
2. Proof = implementation
3. Lemma = helper function

4. Proof strategy = algorithm or design pattern
    - **Equational reasoning**: a chain of justified rewrites
    - **Extensionality and case distinction**
    - **Structural induction**

Like programming, proving takes *practice*

Equational reasoning

`foldr` over a singleton list

If `e` is the two-sided unit of `op`, then `foldr op e [x] = x`

Which half of the unit law will the proof use?

```
foldr op e [x]
= ...
```

`foldr` over a singleton list

If `e` is the two-sided unit of `op`, then `foldr op e [x] = x`

```
foldr op e [x]
= -- rewrite list notation
foldr op e (x : [])
= -- definition of foldr, case cons
op x (foldr op e [])
= -- definition of foldr, case empty
op x e
= -- e is a right unit for op
x
```

`foldl` over a singleton list

If `e` is the two-sided unit of `op`, then `foldl op e [x] = x`

Which half of the unit law will this proof use?

```
foldl op e [x]
= ...
```

Try it yourself!

`foldl` over a singleton list

If `e` is the two-sided unit of `op`, then `foldl op e [x] = x`

```
foldl op e [x]
= -- rewrite list syntactic sugar
foldl op e (x : [])
= -- definition foldl
foldl op (op e x) []
= -- definition foldl
op e x
= -- e is a left unit for op
x
```

Function composition is associative

For all functions `f`, `g` and `h`, `f . (g . h) = (f . g) . h`

90-second task: how should a proof of equality between these two functions proceed?

Proving functions equal

- To prove `f = g`, choose an arbitrary input `x` and prove `f x = g x`
  - This principle is called **extensional equality**
- The implementations may differ; their results must agree on every input
- “Arbitrary” matters
  - Do not assume a special value or shape unless you cover all cases
- In this lecture we apply extensionality to pure functions

Two column style proofs

Reasoning from two ends is typically easier

- Rewrite the expression until you reach the same point
- Equalities can be read “backwards”

For all functions `f`, g and h, `f . (g . h) = (f . g) . h`

Proof: consider any x

```
(f . (g . h)) x
= {- defn. of (.) -}
f ((g . h) x)
= {- defn. of (.) -}
f (g (h x))
```

```
((f . g) . h) x
= {- defn. of (.) -}
(f . g) (h x)
= {- defn. of (.) -}
f (g (h x))
```

`map after (:)`

For `f :: a -> b` and `x :: a`,

```
map f . (x :) = (f x :) . map f
```

map after (:)

For `f :: a -> b` and `x :: a`,

```
map f . (x :) = (f x :) . map f
```

Proof: consider any list `xs`

```
(map f . (x :)) xs
= {- defn of (.) -}
map f ((x :) xs)
= {- section notation -}
map f (x : xs)
= {- defn. of map -}
f x : map f xs
```

```
((f x :) . map f) xs
= {- defn of (.) -}
(f x :) (map f xs)
= {- section notation -}
f x : map f xs
```

`not` is an involution

The functions `not . not` and `id` are equal

Task: name the proof principles before writing the equations.

`not` is an involution

The functions `not . not` and `id` are equal

Proof: by extensionality and case distinction.

- Case `x = False`

```
(not . not) False = not (not False) = False = id False
```

`not` is an involution

The functions `not . not` and `id` are equal

Proof: by extensionality and case distinction.

- Case `x = False`

  ```
  (not . not) False = not (not False) = False = id False
  ```

- Case `x = True`

  ```
  (not . not) True  = not (not True)  = True  = id True
  ```

Each step unfolds `(.)`, `not`, or `id`.

Case distinction

- To prove a property *for all* x, sometimes we need to distinguish the possible shapes that x may take
  - We need to be exhaustive to cover *all* cases

Case distinction

- To prove a property *for all* x, sometimes we need to distinguish the possible shapes that x may take
  - We need to be exhaustive to cover *all* cases

- For example,
  - A Boolean may be either `True` or `False`
  - A `Maybe a` value could be `Nothing` or `Just x` for some x
  - Given a data type of the form

```
data Shape = Circle    Point Float
           | Rectangle Point Float Float
           | Triangle  Point Point Point
```

  you need to consider three different cases

Homework: Booleans and `(&&)` form a monoid

1. `True` is a neutral element: for any Boolean `x`,

   ```
   True && x = x
   x && True = x
   ```

2. `(&&)` is associative: for any Booleans `x`, `y`, and `z`,

   ```
   x && (y && z) = (x && y) && z
   ```

Homework: `Maybe a` forms a monoid

Consider the following operation:

```
Just x  <|> _ = Just x
Nothing <|> y = y
```

1. `Nothing` is a neutral element: for any `x :: Maybe a`,

   ```
   Nothing <|> x = x
   x <|> Nothing = x
   ```

2. `(<|>)` is associative

Induction on data types

The case for lists

- Every finite list is built from `[]` and `(:)`

  `x : (y : (z : ... (w : [])))`

- Suppose that
  - we prove a property P for `[]`;
  - for arbitrary `x` and `xs`, P for `xs` implies P for `x:xs`.
- Structural induction says that P then holds for **every finite list**
  - Infinite lists require separate coinductive reasoning, which we do not cover in this course.

The case for numbers and trees

- Every finite natural number can be seen as applying the successor function finitely many times to 0

```
4 = Succ (Succ (Succ (Succ Zero)))
```

  - What if...?
    - we prove a property P for 0
    - given a number n satisfying P, we can prove P for `succ n = n + 1`

The case for numbers and trees

- Every finite natural number can be seen as applying the successor function finitely many times to 0

  ```
  4 = Succ (Succ (Succ (Succ Zero)))
  ```

  - What if...?
    - we prove a property P for 0
    - given a number n satisfying P, we can prove P for `succ n = n + 1`

- Every (finite) binary tree is built by finitely many Nodes ultimately applied to `Leaf`
  - What if...?
    - we prove a property P for `Leaf`
    - given any two trees `l` and `r` satisfying P and a value `x`, we can prove P for `Node l x r`

Structural induction

A strategy for proving properties of structured data

1. State the law and introduce its input variables
2. Choose one variable and make one case per constructor
3. Prove constructors without recursive fields directly
4. For each recursive constructor
    a. state one *induction hypothesis* (IH) per recursive field;
    b. unfold definitions until an IH applies.

Structural induction for lists

1. State the law and choose the induction variable
    - Prefer an argument consumed by the relevant recursive definitions
2. Prove the case `[]` by equational reasoning
3. For `x:xs`, state the induction hypothesis for `xs`
4. Unfold definitions until the induction hypothesis applies

`map f` distributes over `(++)`

For all lists `xs` and `ys`

```
map f (xs ++ ys) = map f xs ++ map f ys
```

Before proving it: should we induct on `xs` or `ys`?

Inspect which argument the definition of `(++)` consumes.

map f distributes over (++)

For all lists xs and ys

```
map f (xs ++ ys) = map f xs ++ map f ys
```

Proof: by induction on xs

- Case xs = []

```
map f ([] ++ ys)
= {- defn. of (++) -}
map f ys
```

```
map f [] ++ map f ys
= {- defn. of map -}
[] ++ map f ys
= {- defn of (++) -}
map f ys
```

map f distributes over (++)

- Case `xs = z:zs`
  - IH: `map f (zs ++ ys) = map f zs ++ map f ys`

```
map f ((z : zs) ++ ys)
= {- defn. of (++) -}
map f (z : (zs ++ ys))
= {- defn of map -}
f z : map f (zs ++ ys)
```

```
  map f (z : zs) ++ map f ys
= {- defn. of map -}
(f z : map f zs) ++ map f ys
= {- defn of (++) -}
f z : (map f zs ++ map f ys)
= {- IH, right-to-left -}
f z : map f (zs ++ ys)
```

`map` preserves composition

For all compatible functions `f` and `g`,
`map (f . g) = map f . map g`

Proof: two reasoning layers are needed

1. By extensionality, prove for every `xs` that
   `map (f . g) xs = (map f . map g) xs`

map preserves composition

For all compatible functions `f` and `g`,
`map (f . g) = map f . map g`

Proof: two reasoning layers are needed

1. By extensionality, prove for every `xs` that
   `map (f . g) xs = (map f . map g) xs`
2. Prove that list equality by induction on `xs`

- Case `xs = []`

```
map (f . g) []          (map f . map g) []
= {- defn. of map -}    = {- defn of (.) -}
[]                      map f (map g [])
                        = {- defn. of map, twice -}
                        []
```

map preserves composition

- Case `xs = z:zs`
  - IH: `map (f . g) zs = (map f . map g) zs`

```
map (f . g) (z : zs)
= {- defn. of map -}
(f . g) z : map (f . g) zs
= {- defn of (.) -}
f (g z) : map (f . g) zs
```

```
(map f . map g) (z : zs)
= {- defn. of (.) -}
map f (map g (z : zs))
= {- defn. of map -}
map f (g z : map g zs)
= {- defn. of map -}
f (g z) : map f (map g zs)
= {- IH -}
f (g z) : map (f . g) zs
```

`reverse` is an involution

The functions `reverse . reverse` and `id` are equal

Proof: by extensionality we need to prove that for all `xs`

```
(reverse . reverse) xs
= reverse (reverse xs)    =  id xs
```

`reverse` is an involution

The functions `reverse . reverse` and `id` are equal

Proof: by extensionality we need to prove that for all `xs`

```
(reverse . reverse) xs
= reverse (reverse xs)    =  id xs
```

We proceed by induction on `xs`

- Case `xs = []`

```
reverse (reverse [])        id []
= {- defn. of reverse -}    = {- defn. of id -}
reverse []                  []
= {- defn. of reverse -}
[]
```

`reverse` is an involution

- Case `xs = z:zs`
  - IH: `reverse (reverse zs) = zs`

```
reverse (reverse (z : zs))      id (z : zs)
= {- defn. of reverse -}      = {- defn of id -}
reverse (reverse zs ++ [z])   z:zs
```

The IH is hidden inside `reverse (reverse zs ++ [z])`.

60-second task: what lemma would expose it?

Lemmas

To keep going we defer some parts as *lemmas*

- Similar to local definitions in code
- Lemmas have to be proven separately

The main missing lemma is

```
-- reverse reverses the order of append
reverse (xs ++ ys) = reverse ys ++ reverse xs
```

We also use `reverse [x] = [x]`.

Proving the append lemma uses two familiar laws: `ws ++ [] = ws` and associativity of `(++)`.

`reverse` is an involution

```
reverse (reverse (z : zs))
= {- defn. of reverse -}
reverse (reverse zs ++ [z])
= {- reverse over (++) -}
reverse [z] ++ reverse (reverse zs)
= {- singleton lemma -}
[z] ++ reverse (reverse zs)
= {- IH -}
[z] ++ zs                    id (z : zs)
= {- defn of (++) -}         = {- defn of id -}
z : zs                       z : zs
```

We still need to prove the lemmas separately

`reverse` is an involution

Lemma: `reverse (xs ++ ys) = reverse ys ++ reverse xs`

Proof: by induction on `xs`, using right identity and associativity of `(++)`

Lemma: `reverse [x] = [x]`

Proof:

```
reverse [x]
= {- list notation -}
reverse (x : [])
= {- defn. of reverse -}
reverse [] ++ [x]
= {- defn. of reverse -}
[] ++ [x]
= {- defn. of (++) -}
[x]
```

Mathematical induction

- To prove that a statement P holds for all $n \in \mathbb{N}$
  - Prove that it holds for 0
  - Prove that it holds for $n + 1$ assuming that it holds for n
- This strategy is equivalent to structural induction on

  ```
  data Nat = Zero | Succ Nat
  ```

  This encoding is called *Peano numbers*

Note: there are stronger forms of induction for natural numbers, but we restrict ourselves to the simpler one

Arithmetic using Peano numbers

Addition and multiplication are defined by recursion

```
add  :: Nat -> Nat -> Nat
add  Zero     m = m
--        0 + m = m
add  (Succ n) m = Succ (add n m)
--  (n + 1) + m = (n + m) + 1

mult :: Nat -> Nat -> Nat
mult Zero     m = Zero
--        0 × m = 0
mult (Succ n) m = add (mult n m) m
--  (n + 1) × m = (n × m) + m
```

0 is right identity for addition

For all natural n, `add n Zero = n`

Proof: by induction on n

- Case `n = Zero`

```
add Zero Zero
= {- defn. of add -}
Zero
```

- Case `n = Succ p`
  - IH: `add p Zero = p`

```
add (Succ p) Zero
= {- defn. of add -}
Succ (add p Zero)
= {- IH -}
Succ p
```

Some functions over binary trees

```
data Tree a = Leaf | Node (Tree a) a (Tree a)
```

`size t` counts the number of nodes

```
size Leaf         = 0
size (Node l _ r) = 1 + size l + size r
```

`mirror t` swaps the left and right subtrees at every node

```
mirror Leaf         = Leaf
mirror (Node l x r) = Node (mirror r) x (mirror l)
```

`mirror` preserves the size

For all trees `t`, `size (mirror t) = size t`

`mirror` preserves the size

For all trees `t`, `size (mirror t) = size t`

Proof: by induction on `t`

- Case `t = Leaf`

```
size (mirror Leaf)
= {- defn. of mirror -}
size Leaf
```

`mirror` preserves the size

- Case `t = Node l x r`
  - We get one induction hypothesis per recursive position
  - IH1: `size (mirror l) = size l` IH2: `size (mirror r) = size r`

```
size (mirror (Node l x r))
= {- defn. of mirror -}
size (Node (mirror r) x (mirror l))
= {- defn. of size -}
1 + size (mirror r) + size (mirror l)
= {- IH1 and IH2 -}
1 + size r + size l
= {- associativity and commutativity of addition -}
1 + size l + size r
= {- defn. of size -}
size (Node l x r)
```

Exit ticket: one more induction

Prove `mult n Zero = Zero` for every `n :: Nat`

1. State the induction variable and induction hypothesis.
2. Show exactly where the induction hypothesis is used.

Extension: prove `mirror (mirror t) = t`.

Summary

- Program equivalences support both *correctness* and *performance*
- Match the proof principle to the type:
  - functions: extensional equality;
  - non-recursive data: case distinction;
  - recursive data: structural induction.
- Choose an induction variable by inspecting recursive definitions
- If a proof gets stuck, look for a stronger statement or an auxiliary lemma

Some advice

- Before calculating, write the law, cases, and induction hypotheses
- Label every rewrite; remember that equalities may be used in either direction
- Proving takes practice, just like programming
  - The book and lecture notes contain many more examples
- Inductive proofs are *definitely* part of the final exam
  - They may concern lists, natural numbers, trees, or another recursive data type